PROJECT 2

CS 180 · COMPUTATIONAL PHOTOGRAPHY

FUN WITH FILTERS AND FREQUENCIES

I treated images as signals: I built convolution from scratch and used derivative filters to find edges, then used Gaussian low- and high-pass filtering to sharpen photos, make hybrid images that change with viewing distance, and blend photos seamlessly one frequency band at a time.

by **Sohum Phadke** / Fall 2026 / assignment

CONTENTS

9 ITEMS

1.1 Convolutions from scratch
1.2 Finite difference operator
1.3 Derivative of Gaussian
B&W: gradient orientation
2.1 Sharpening
2.2 Hybrid images
2.3 Gaussian & Laplacian stacks
2.4 Multiresolution blending
What I learned

PART 1 · FUN WITH FILTERS

1.1 CONVOLUTIONS FROM SCRATCH

Convolution slides a small filter over the image. At every output pixel, it multiplies the filter against that pixel's neighborhood and adds up the products. The filter is flipped 180° first (that flip is what makes it convolution rather than correlation). To keep the output the same size as the input, the image is zero-padded on every side by half the filter size.

FOUR LOOPS: OVER OUTPUT PIXELS (I, J), THEN OVER FILTER ENTRIES (U, V)

```
def conv_four_loops(img, kernel):
    kernel = kernel[::-1, ::-1]
    kh, kw = kernel.shape
    ph, pw = kh // 2, kw // 2
    padded = np.pad(img, ((ph, ph),
    H, W = img.shape
    out = np.zeros((H, W))
    for i in range(H):
        for j in range(W):
            total = 0
            for u in range(kh):
                for v in range(kw):
                    total += padded
            out[i, j] = total
    return out
```

TWO LOOPS: SLICE THE NEIGHBORHOOD, MULTIPLY ELEMENT-WISE, SUM

```
def conv_two_loops(img, kernel):
    kernel = kernel[::-1, ::-1]
    kh, kw = kernel.shape
    padded = np.pad(img, ((kh//2, (
                          (kw//2, (
    H, W = img.shape
    out = np.zeros((H, W))
    for i in range(H):
        for j in range(W):
            patch = padded[i:i+kh,
            out[i, j] = np.sum(patc
    return out
```

Both match `scipy.signal.convolve2d(img, k, mode="same", boundary="fill")` to floating-point precision (`np.allclose` is true for every filter). Runtimes on my photo:

kernel	4 loops	2 loops	scipy	matches scipy
box	0.534 s	0.077 s	3.30 ms	yes
Dx	0.024 s	0.076 s	0.22 ms	yes

Dy	0.027 s	0.076 s	0.34 ms	yes

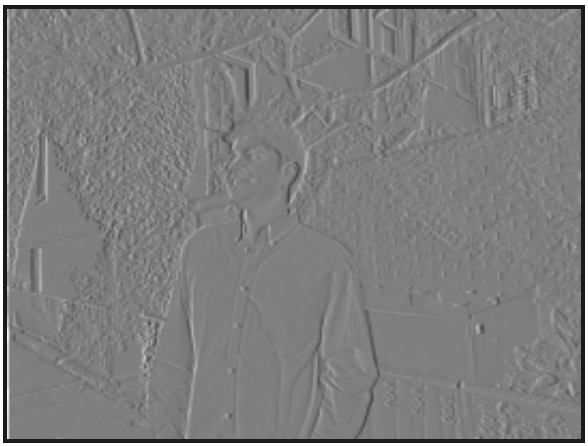
Image size: 200 × 268 pixels.

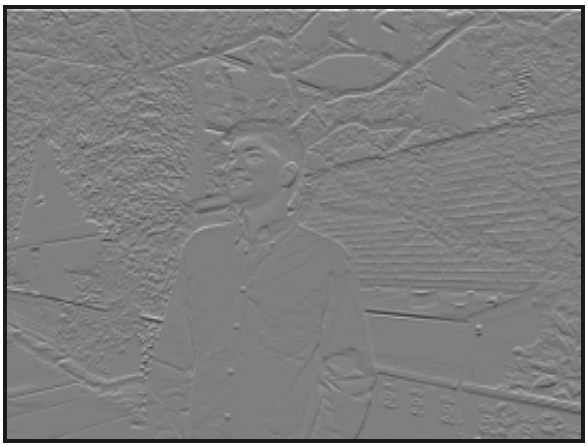
Runtime. scipy is 100–300× faster than either of mine because it runs compiled code instead of Python loops. For the 9×9 box filter, two loops beat four loops by about 7×, since each pixel's 81 multiply-adds happen in one numpy call. For the 2-entry D_x/D_y filters it flips: four loops is about 3× *faster*, because the per-pixel cost of slicing and calling `np.sum` outweighs two plain Python multiplications.

Boundaries. My functions pad with zeros, the same as scipy's `boundary="fill"`, which is why they match. Zero padding acts like a black frame around the picture: the box-filtered image visibly darkens along its border, and the derivative images show a false edge along the frame. scipy also offers `"symm"` (mirror the edge pixels) and `"wrap"`, which avoid this. I use `"symm"` in the rest of the project.

Me, read as grayscale

9×9 box filter: note the dark border from zero padding





$D_x = [1\ -1]$: railing bars and window frames

$D_y = [1\ -1]^T$: brick rows and the wall top

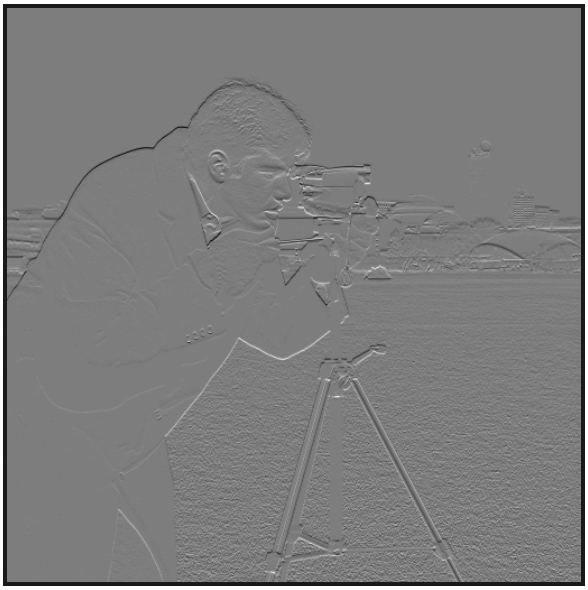
Derivative images use gray = 0, white = positive, black = negative.

1.2 FINITE DIFFERENCE OPERATOR

Convolving the cameraman image with D_x gives the horizontal change in brightness, which lights up vertical edges like the tripod legs. D_y gives the vertical change, which lights up horizontal edges like the horizon and the camera body. The gradient magnitude $\sqrt{(d_x^2 + d_y^2)}$ combines the two into a single "how fast is brightness changing here" image that doesn't depend on edge direction.

Original

$\partial I/\partial x$



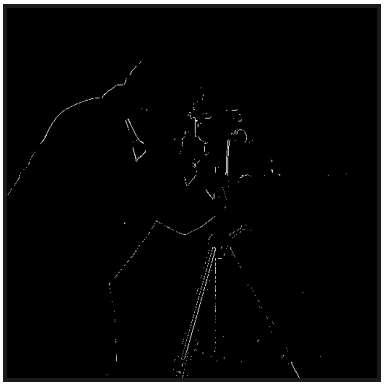
$\partial I/\partial y$

Gradient magnitude

To get an edge image, I keep pixels whose magnitude is above a threshold. The grass is the hard part: its texture produces many small gradients. At 0.15 (on a 0–1 image) the grass becomes speckle; at 0.40 the coat and tripod outlines break apart. I chose **0.25**: the man, camera and tripod stay connected and most of the grass noise is gone. The tradeoff is losing some faint background buildings, which I accepted to keep noise down.

threshold 0.15: grass turns to speckle

threshold 0.25 (chosen)



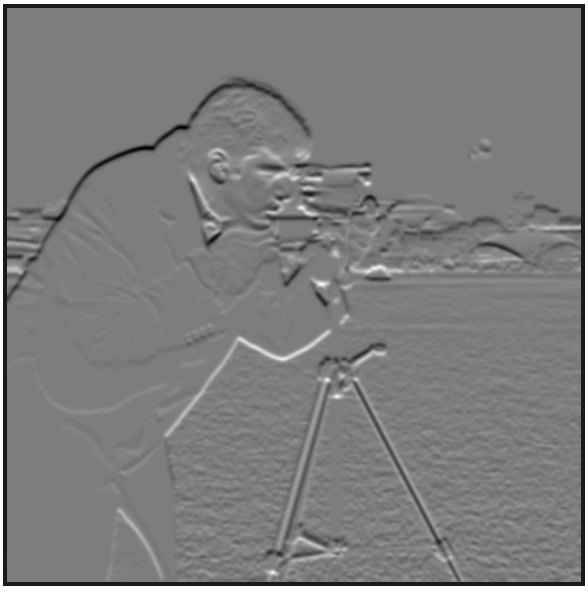
threshold 0.40: real edges break up

1.3 DERIVATIVE OF GAUSSIAN

The finite-difference result is noisy because a derivative amplifies pixel-to-pixel noise. Blurring first with a Gaussian G (σ = 1.5, 11×11, built with `cv2.getGaussianKernel` and an outer product with its transpose) removes that noise before differentiating.

Blurred, σ = 1.5

$\partial/\partial x$ of blurred



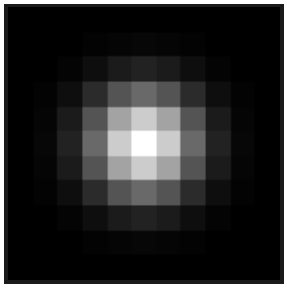
$\partial/\partial y$ of blurred

Gradient magnitude

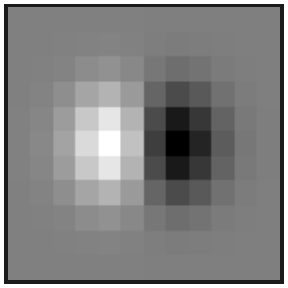
What changed: the edges are smoother, thicker and continuous, and the grass speckle is nearly gone. Smoothing lowers every gradient value, so the threshold drops to **0.07**. The cost is

that very fine detail disappears and edge positions are slightly less precise.

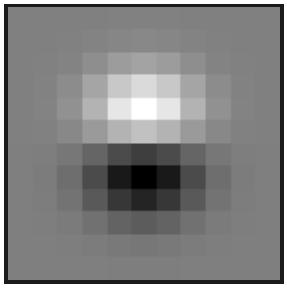
Convolution is associative, so $(I * G) * D_x = I * (G * D_x)$. Building the derivative-of-Gaussian filters once gives the same result with a single convolution per direction:



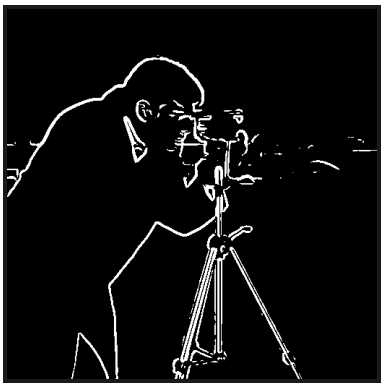
Gaussian G



$DoG_x = G * D_x$

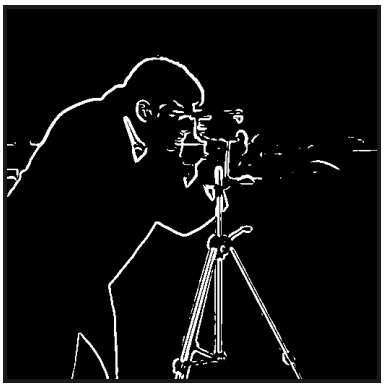


$DoG_y = G * D_y$



Blur, then differentiate (two convolutions)

DoG filters (one convolution): magnitude



DoG filters: edges at 0.07

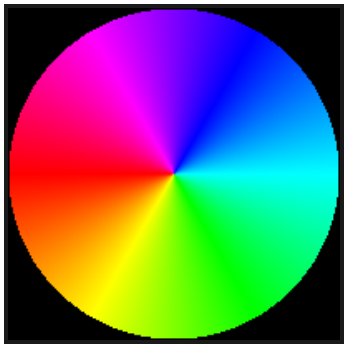
Verified: away from the border (where the two methods pad differently), the largest difference between the two magnitude images is about 10^{-15}, which is floating-point rounding.

B&W: GRADIENT ORIENTATION

At each pixel, the gradient (d_x, d_y) points in the direction brightness increases fastest. I compute its angle as $\theta = \text{atan2}(-d_y, d_x)$ (d_y is negated because image rows count downward) and map $\theta \in [-\pi, \pi]$ onto hue $\in [0, 1]$ in HSV. Hue is circular, so $-\pi$ and π get the same color, which is right for a direction. Saturation is 1 and value is the gradient magnitude, so flat regions stay dark and only edges are colored. The two sides of a thin object, like a tripod leg, get opposite colors because the gradient flips direction across it.

Hue = orientation, brightness = magnitude

Thresholded edges only



Key: color for each gradient direction

PART 2 · FUN WITH FREQUENCIES

2.1 SHARPENING

A Gaussian blur keeps only the low frequencies, so subtracting it from the image leaves the high frequencies: edges and texture. Adding a multiple α of those back makes the image look sharper:

$$f_{sharp} = f + \alpha(f - f * G) = f * ((1 + \alpha)\delta - \alpha G)$$

where δ is the unit impulse. The right-hand side is one filter, the **unsharp mask**, so sharpening takes a single convolution. It matches the two-step version to within 10^{-15}. I used $\sigma = 2$ and $\alpha = 1.5$.

TAJ MAHAL

Original

Low frequencies (blur, $\sigma = 2$)

High frequencies (original - blur)

Sharpened, α = 1.5

The sharpened version brings out the marble carvings, the edges of the domes, and the row of trees. Increasing α strengthens edges further until bright and dark halos appear along them.

α = 0.5

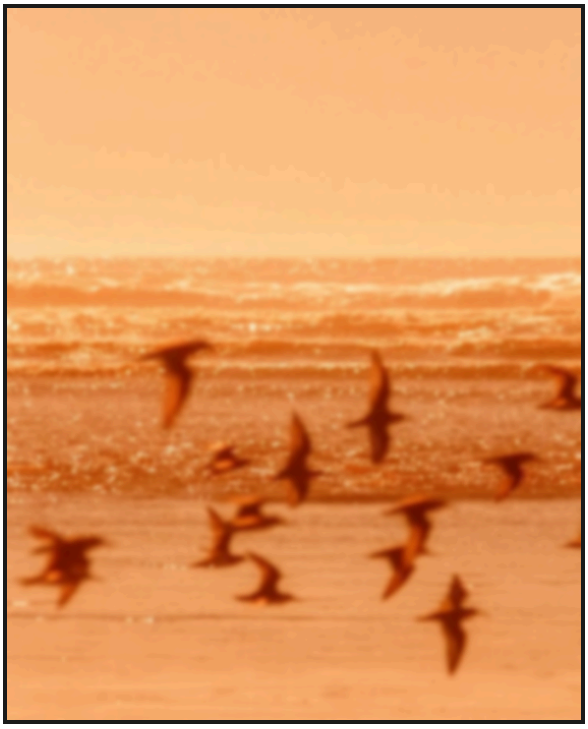
α = 1

α = 2

α = 4: halos, over-sharpened

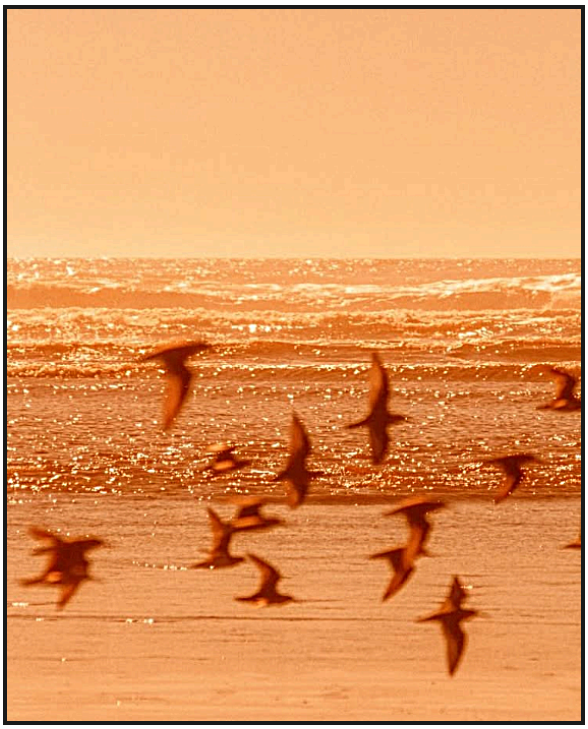
MY IMAGE: BIRDS OVER THE BEACH

Original (soft focus)



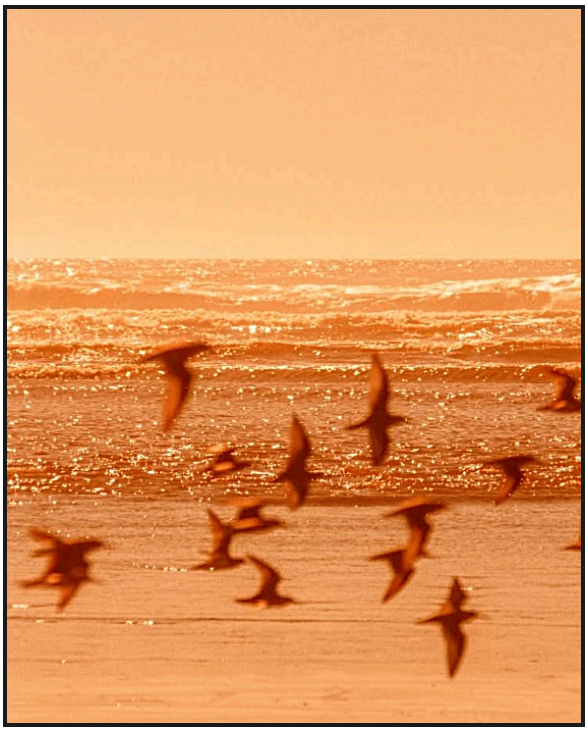
Low frequencies

High frequencies

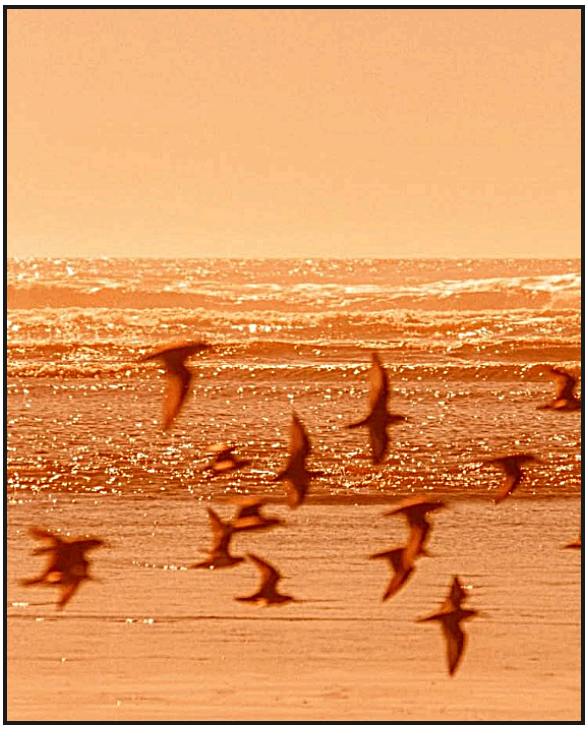


Sharpened, α = 1.5

α = 0.5



α = 1



α = 2

α = 4

The bird silhouettes get darker, crisper outlines, and the glints on the water pop. The birds never become truly sharp, though: they were out of focus to begin with, and sharpening can only boost edges that are still in the image. It can't recreate detail that was never captured.

EVALUATION: BLUR A SHARP PHOTO, THEN SHARPEN IT AGAIN

Original (sharp)

Blurred, σ = 2

Re-sharpened, α = 2

The re-sharpened photo looks much crisper than the blurred one: window frames, bricks and the fire escapes regain contrast. Compared with the original, though, it is not a recovery. The thin railing bars, the small graffiti lettering and the brick mortar lines were erased by the blur, and nothing can bring them back. Sharpening also adds faint halos around the high-contrast edges that the original never had.

> Blurring *removes* high frequencies; sharpening only *amplifies* the ones that survived.

2.2 HYBRID IMAGES

Following Oliva, Torralba & Schyns (2006): up close, high frequencies dominate what we see; from far away, only low frequencies are visible. A hybrid adds the **low frequencies of one image** (Gaussian blur, σ_{low}) to the **high frequencies of another** (image minus its Gaussian blur, σ_{high}). I align each pair first by clicking the two eyes in both images and warping the second image with a similarity transform (rotation, scale and translation).

DEREK + NUTMEG: FULL PROCESS

Derek, original

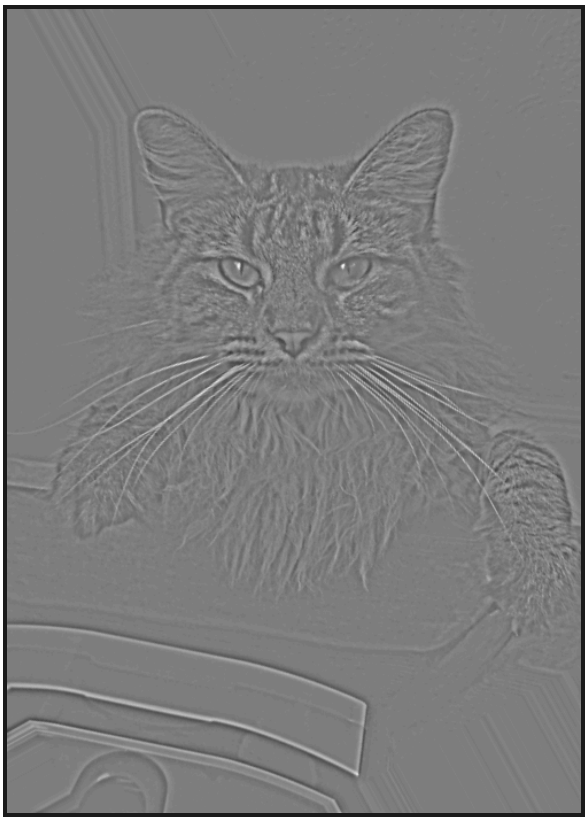
Nutmeg, original (before alignment)

I clicked both eyes in each photo, then rotated, scaled and shifted Nutmeg so her eyes land on Derek's. The aligned pair and the filtered versions:

Derek (seen from far)

Nutmeg, aligned (seen up close)

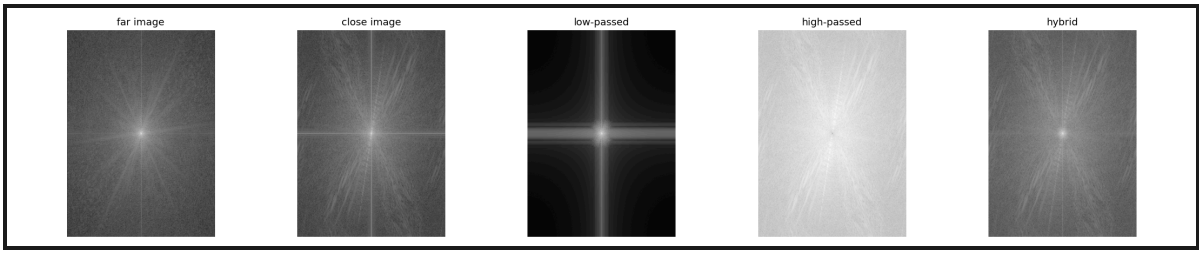
Low-pass of Derek, $\sigma = 6$



High-pass of Nutmeg, $\sigma = 3$

Hybrid

Same hybrid, small: simulates viewing from far away



Log magnitude of the 2D Fourier transform at each step

Frequency analysis. Both inputs have energy spread across all frequencies, with the bright center being the low frequencies. The low-passed Derek keeps only a small bright blob in the center. The high-passed Nutmeg has its center suppressed and keeps the outer spread. The hybrid's spectrum is the two combined: Derek's center plus Nutmeg's outer ring.

Cutoff frequencies. A Gaussian with standard deviation σ falls to half strength at about $0.19/\sigma$ cycles per pixel. With

σ_{low} = 6, Derek keeps frequencies below roughly 0.031 cycles/pixel; with σ_{high} = 3, Nutmeg keeps those above roughly 0.062. I picked these by trying values: a smaller σ_{low} let Derek's features show through up close, and a larger σ_{high} let the cat's fur overwhelm Derek even from a distance. Leaving a gap between the two cutoffs keeps the two images from competing in the middle frequencies.

ME, NOW AND OLDER

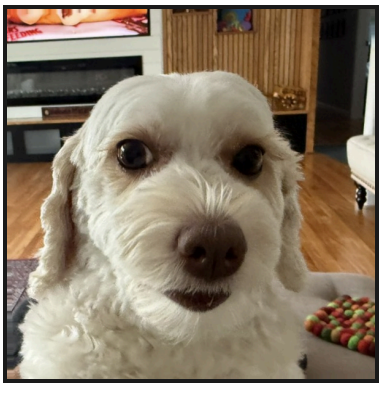
Now (seen from far)

Older (seen up close)

Hybrid, σ_{low} = 6, σ_{high} = 3

This pair works especially well because the pose, hoodie and beach match almost exactly. Up close, the gray hair texture and smile lines dominate. From far away those fine details vanish and the smoother, younger face takes over. It's a change over time.

MY DOG: CURIOUS VS. GRINNING



Grinning (seen from far) | Curious (seen up close) | Hybrid, $\sigma_{low} = 6$, $\sigma_{high} = 3$

A change of expression. The dog was much closer to the camera in one photo, so I cropped both to the same head framing before aligning. Otherwise, the large rescale smeared the image edges. Up close, the fur texture and the wide-eyed, closed-mouth look come through. From a distance, the grin's dark mouth shape takes over.

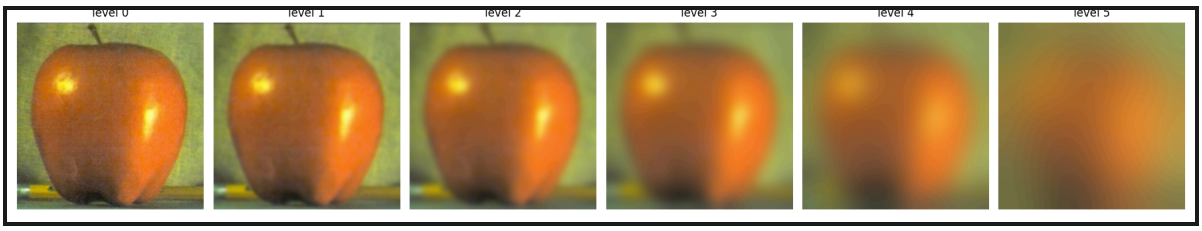
BELLS & WHISTLES: COLOR

Color in the low frequencies only | Color in the high frequencies only | Color in both

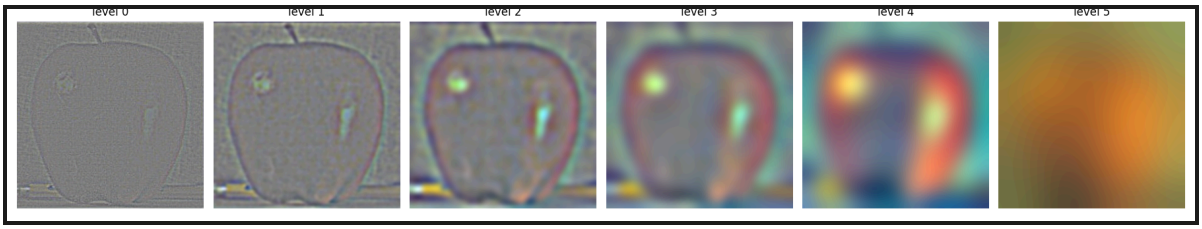
Color works best in the **low-frequency** image. Color mostly lives in large, smooth regions, which is exactly what a low-pass keeps, so it strengthens the far-away interpretation (Derek's skin tone) without interfering up close. In the high-pass image, color survives only as thin fringes along edges, so it adds very little. Using color in both gives the richest image, but the low-frequency color then dominates and makes the close-up cat harder to see.

2.3 STACKS

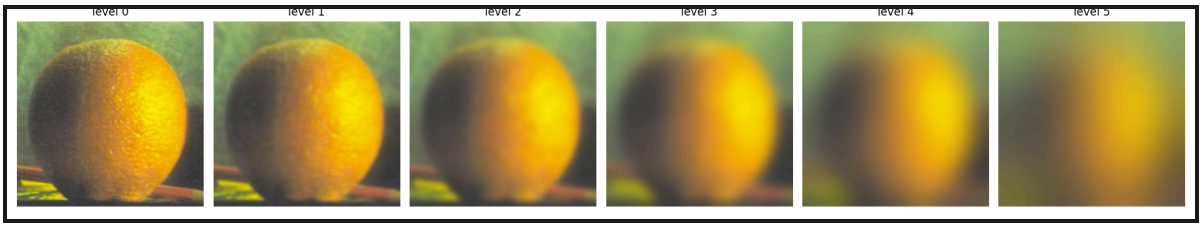
A stack is a pyramid without downsampling: every level stays full size. Each Gaussian level blurs the previous one, with σ doubling each level (2, 4, 8, 16, 32) to mimic a pyramid halving its resolution. Each Laplacian level is the difference between two neighboring Gaussian levels, so it holds one band of frequencies. The last Laplacian level is the final Gaussian level, so summing all levels rebuilds the original exactly. The stacks are implemented from scratch; no built-in pyramid functions are used.



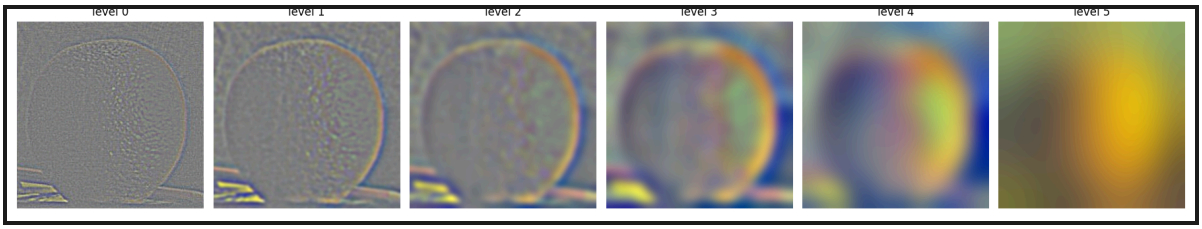
Gaussian stack: apple



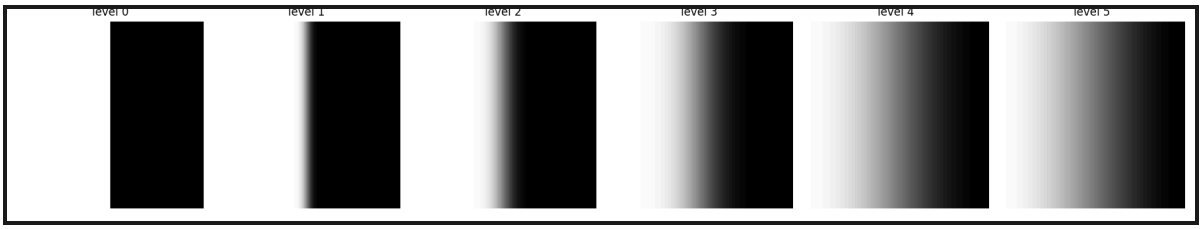
Laplacian stack: apple (band-pass levels shown with gray = 0)



Gaussian stack: orange



Laplacian stack: orange



Gaussian stack of the mask: the seam widens at each level

RECREATING FIGURE 3.42 (SZELISKI)

Rows show Laplacian levels 0, 2 and 4 and the summed result. Columns show the masked apple, the masked orange and their sum. At level 0 (fine detail) the switch between fruits is sharp; by level 4 (coarse color) it spreads across a wide band. That is why the final seam is invisible.

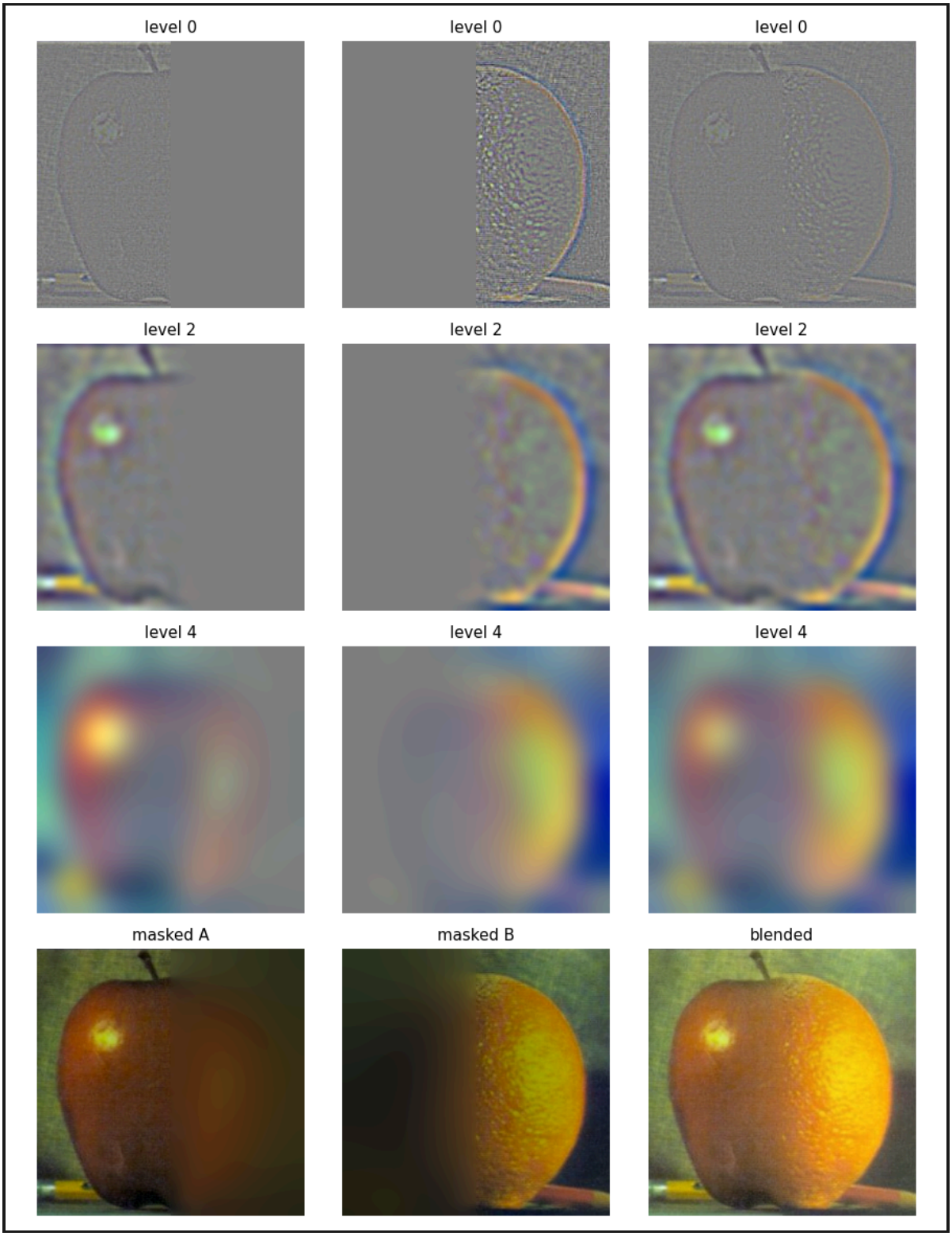


Figure 3.42 (a)-(l), recreated with stacks

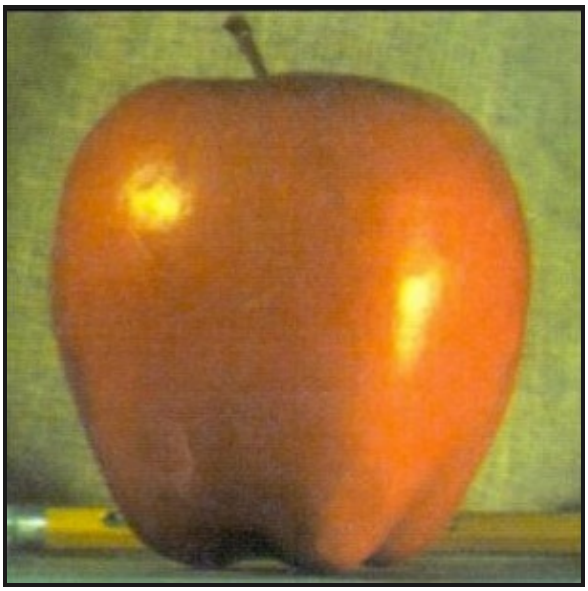
2.4 MULTIRESOLUTION BLENDING

Following Burt & Adelson (1983), each frequency band is blended with a matching blur of the mask:

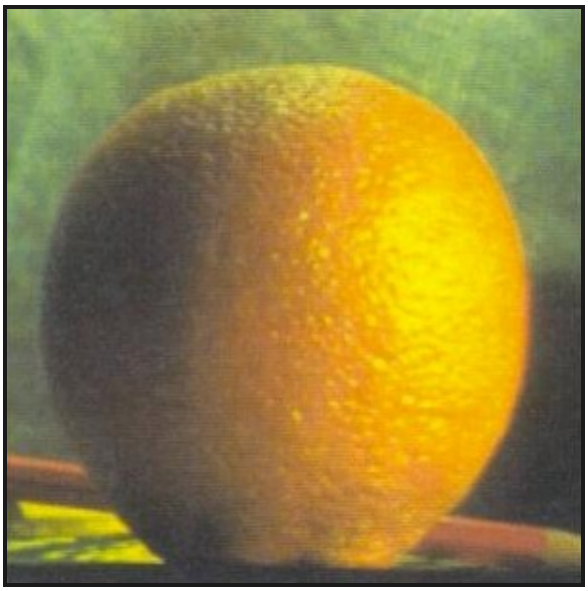
$$\text{blend} = \Sigma_i \ GM_i \cdot LA_i + (1 - GM_i) \cdot LB_i$$

where LA and LB are the Laplacian stacks of the two images and GM is the Gaussian stack of the mask (σ starting at 8). Fine details switch sharply at the seam while smooth color is mixed over a wide band. With stacks, the mask must be blurred explicitly; there is no downsampling to soften it. Each blend below is shown next to a hard paste with the same mask for comparison.

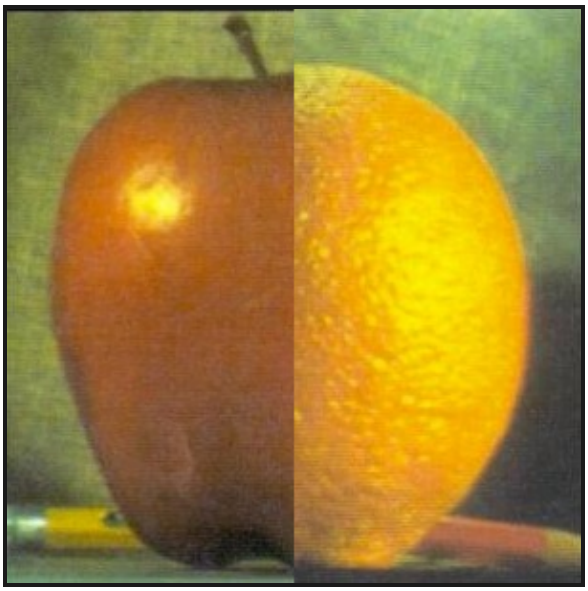
THE ORAPLE (VERTICAL SEAM)



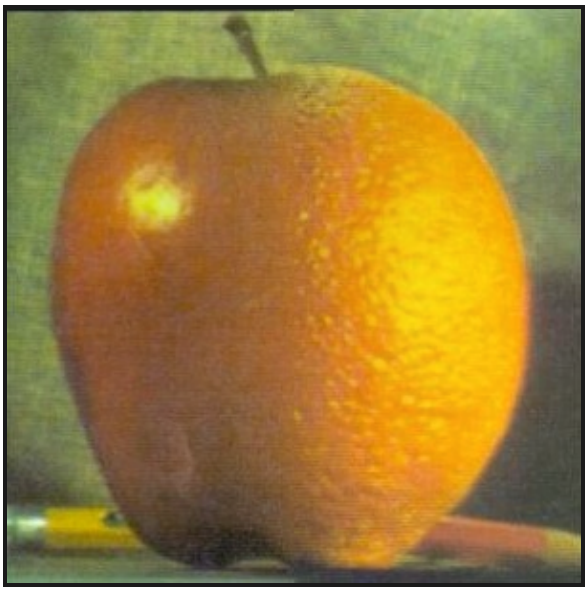
Apple



Orange

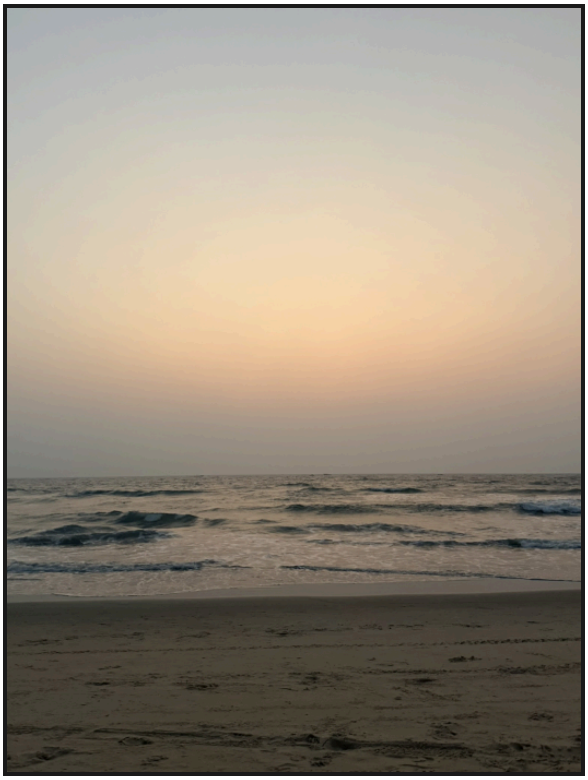


Hard paste: visible seam

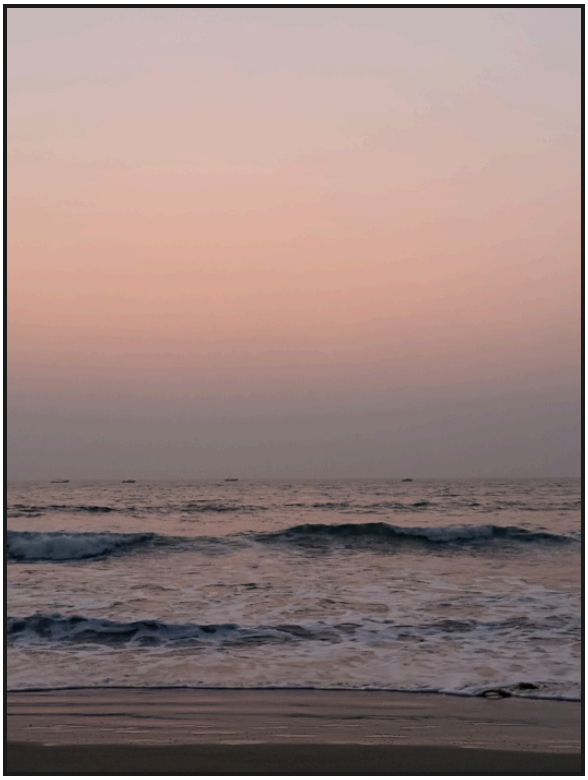


Multiresolution blend

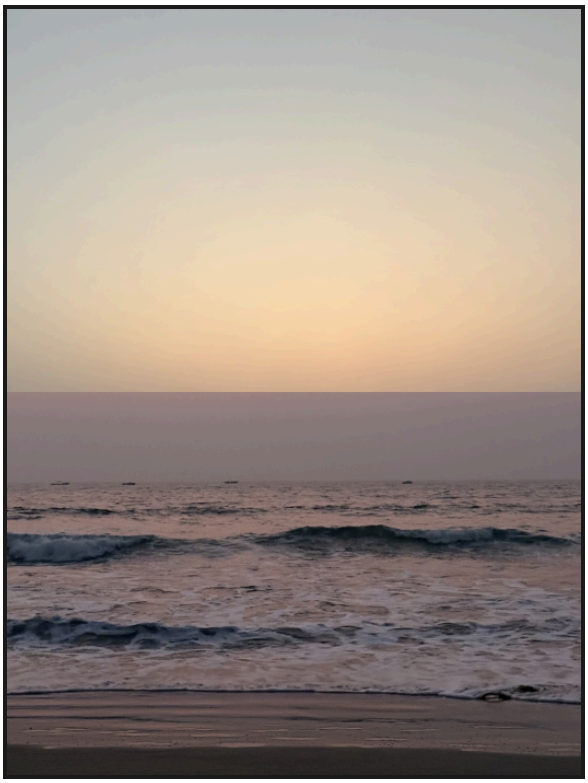
MORNING AND EVENING AT THE SAME BEACH (HORIZONTAL SEAM)



Morning (top)



Evening (bottom)

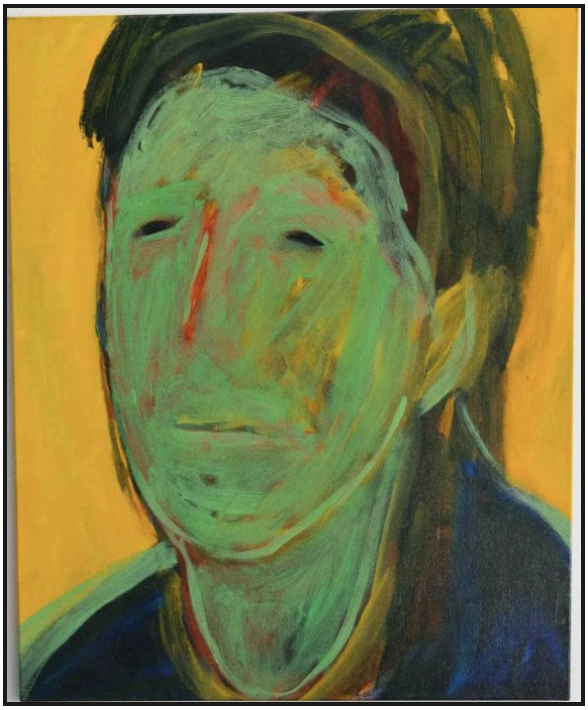


Hard paste

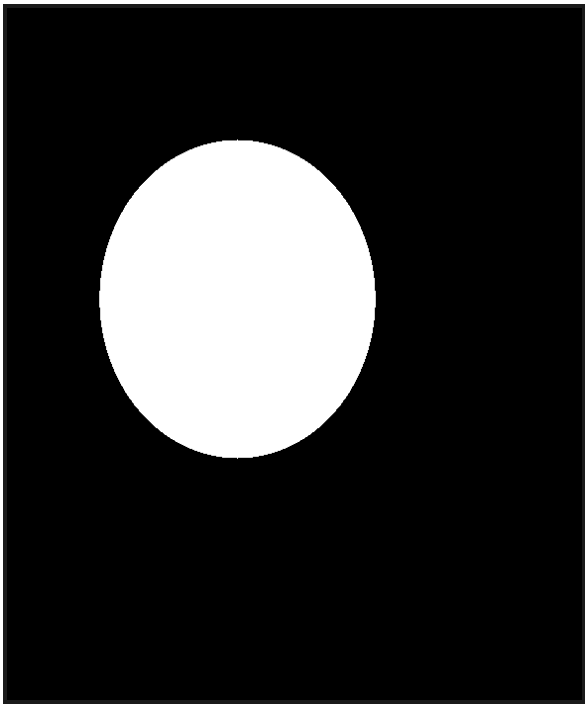
Multiresolution blend

The hard paste leaves a sharp line across the sky where peach meets pink. The blend fades the morning sky into the evening sky so smoothly that it reads as a single sunset photo. Because the horizon sits at the same height in both photos, the sea and sky line up naturally.

MY FACE IN A PAINTING (IRREGULAR MASK)

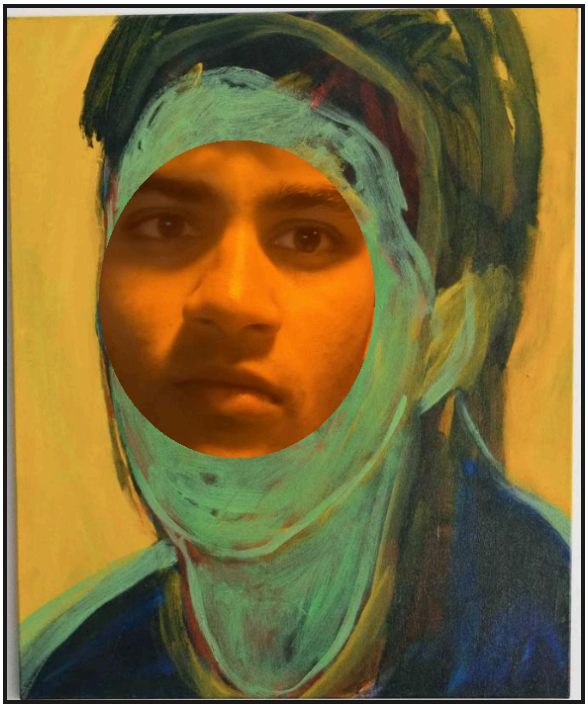


My selfie, aligned so the eyes match the painting

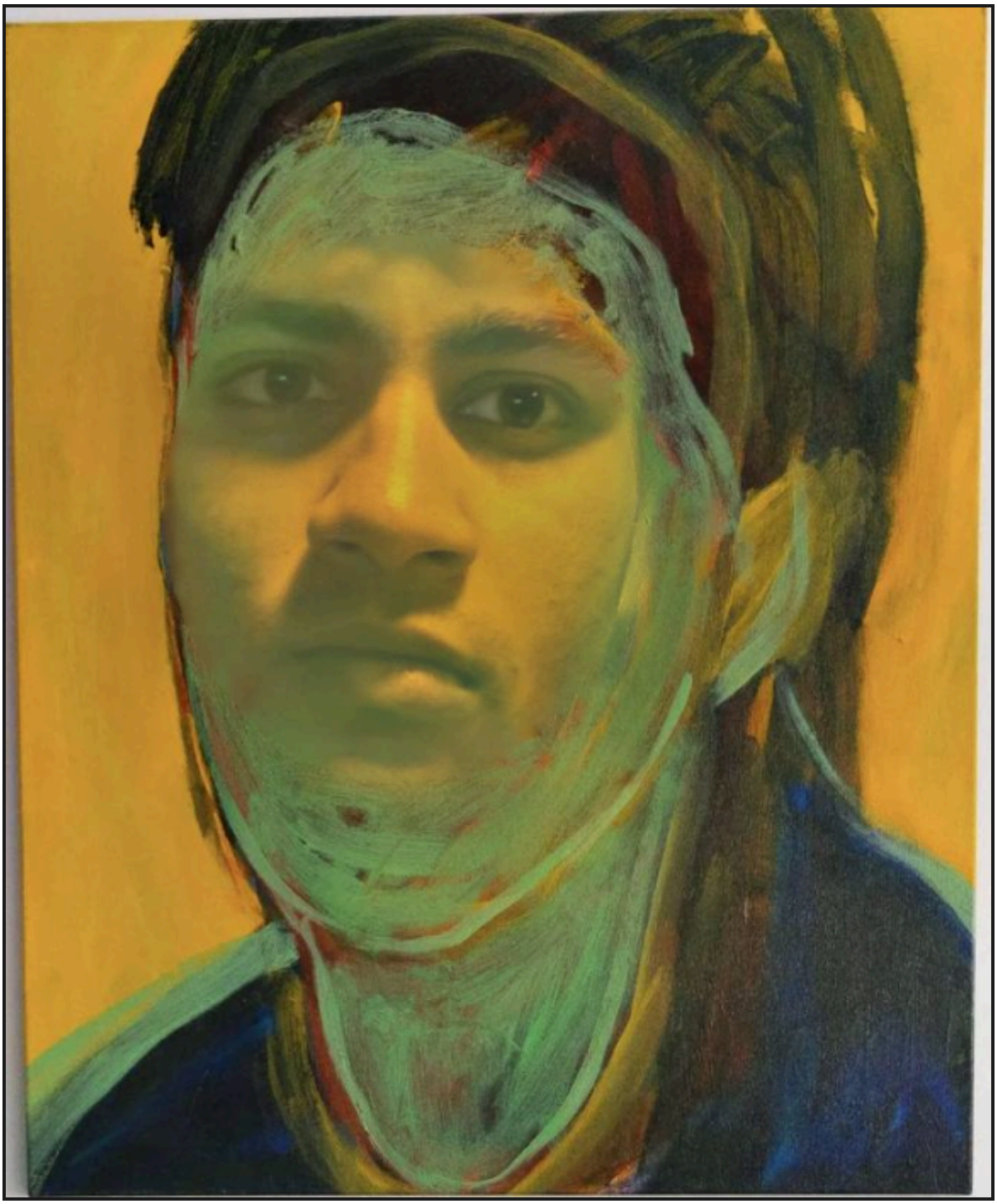


Elliptical mask

Painting



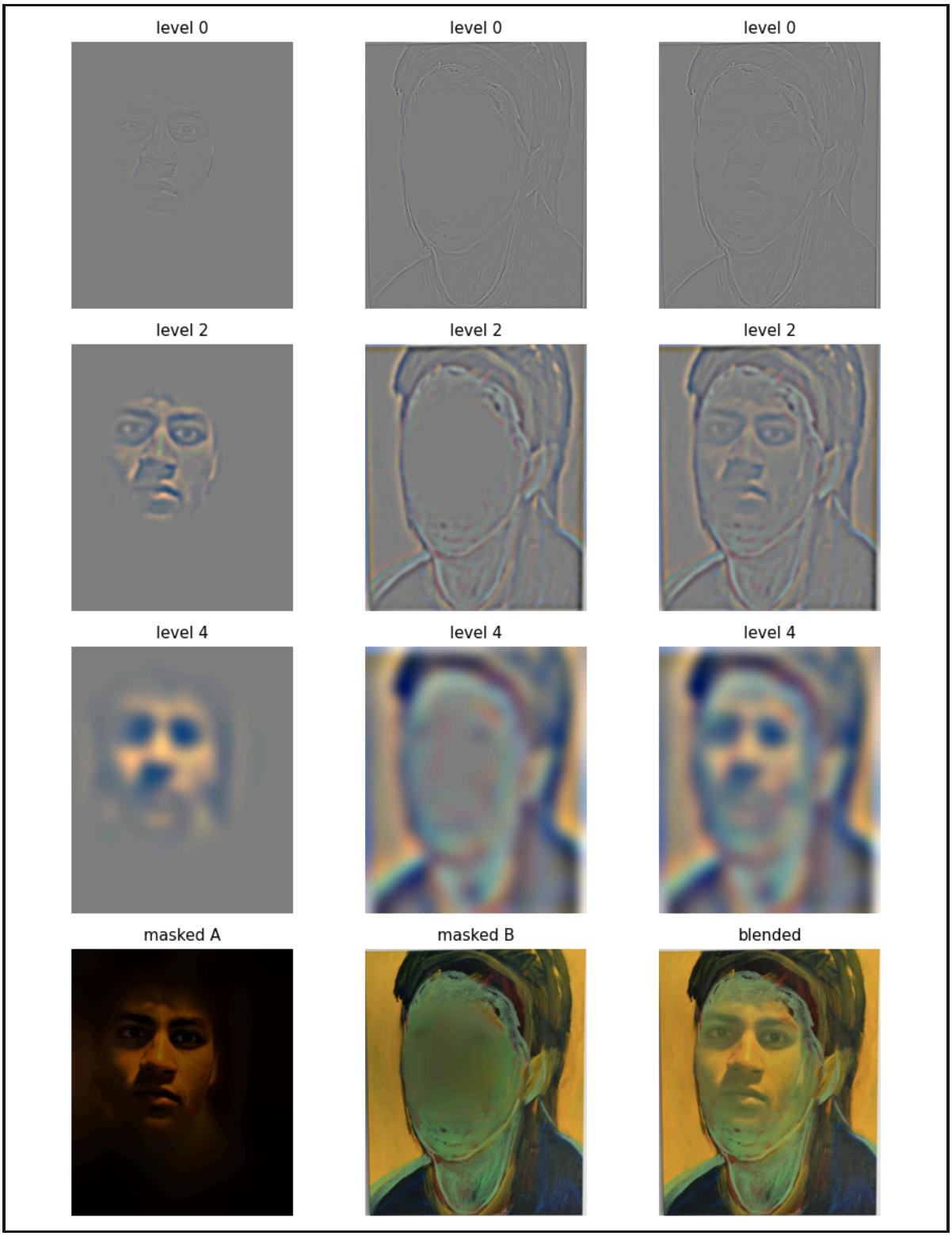
Hard paste



Multiresolution blend

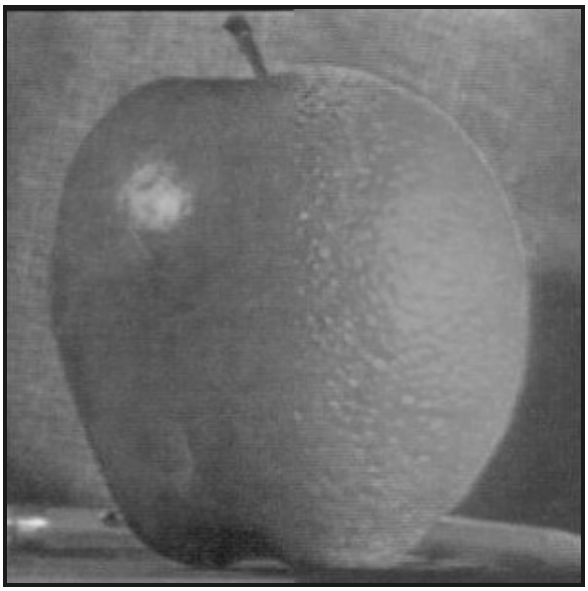
I scaled and positioned the selfie so my eyes land on the painted eyes, then used an ellipse around the face as the mask. The hard paste shows an obvious oval "sticker." In the blend, the painting's green seeps into the edges of my face and fades toward the center. That's the low-frequency color being mixed over a wide band, while my eyes, nose and mouth stay sharp because the high-frequency levels switch at the mask boundary.

LAPLACIAN STACK OF MY FAVORITE BLEND (FIGURE 10 STYLE)

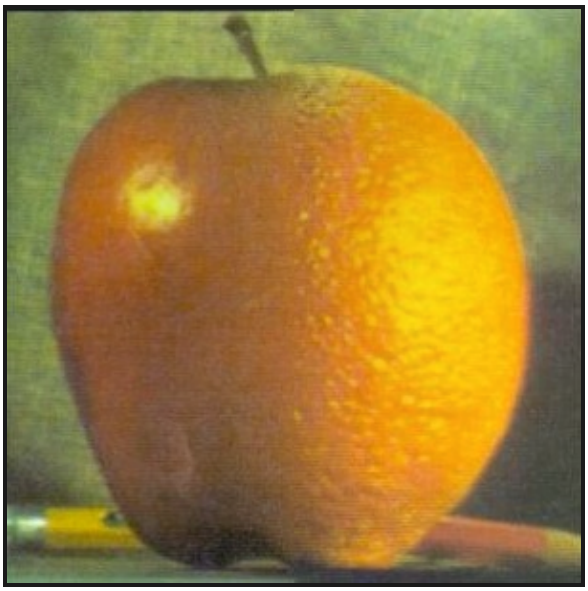


Rows: Laplacian levels 0, 2, 4 and the sum. Columns: masked selfie, masked painting, blend.

BELLS & WHISTLES: COLOR



Grayscale blend



Color blend: each RGB channel blended with the same mask stack

Blending each RGB channel separately makes the results much more convincing. The apple-red to orange-orange hue shift transitions as smoothly as the brightness does, because color differences live mostly in the low frequency levels, which get the widest transition. The painting blend depends on this entirely: in grayscale the green tint that ties my face into the painting would be lost.

WHAT I LEARNED

The most important thing I learned is that an image is a mix of low and high frequencies, and that almost everything in this project comes down to separating and recombining them. Low frequencies carry the smooth parts of a picture, like overall shape, color and lighting; high frequencies carry the edges and fine texture. A single Gaussian blur splits the two apart: the blur keeps the lows, and subtracting it from the original leaves the highs. Once I understood that, sharpening, hybrid images and blending stopped feeling like three separate tricks. Sharpening adds extra highs back in, a hybrid image takes the lows from one photo and the highs from another, and multiresolution blending mixes each frequency band with its own amount of blur on the mask.

What surprised me most was how directly this matches the way we see. My young and older photos turn into a completely different picture depending on how far away I stand, because my eyes stop resolving fine detail at a distance. The painting blend made the same idea visible in a different way: my eyes, nose and mouth stay sharp while the painting's green seeps into the edges of my face, because the fine details switch at the mask boundary while the color fades across a wide band.

I also learned that the algorithm is only half of the result. My first dog hybrid looked terrible until I cropped both photos to the same framing, and my motion-blurred photos barely improved with sharpening, because sharpening can only amplify edges that are still there. It can't recover detail the camera never captured. Choosing good input images, aligning them carefully and tuning values like σ and the edge threshold by eye mattered just as much as writing correct code. Even the convolution itself taught me this: my from-scratch loops gave exactly the same answer as scipy but ran

hundreds of times slower, which made it clear why real image processing relies on optimized libraries.

CS 180 — Computational Photography

← back to all projects